NUMPY ALL METHODS AND CONDITIONS

```
import numpy as np
```

```
arr_1d = np.array([1,2,3])
print("1d array: ", arr_1d)
```

```
1d array:  [1 2 3]
```

```
arr_2d = np.array([1,2,3], [4,5,6])
print("2d array: ", arr_2d)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1735/4060143418.py in <cell line: 0>()
----> 1 arr_2d = np.array([1,2,3], [4,5,6])
      2 print("2d array: ", arr_2d)

TypeError: Field elements must be 2- or 3-tuples, got '4'
```

Next steps: Explain error

```
arr_2d = np.array([[1,2,3], [4,5,6]])
print("2d array: ", arr_2d)
```

```
2d array:  [[1 2 3]
 [4 5 6]]
```

List vs numpy

```
py_list = [1,2,3]
print("python list multiplication ", py_list*2)
```

```
python list multiplication  [1, 2, 3, 1, 2, 3]
```

```
py_array = np.array([1,2,3]) #elelment wise multiplication
print("python array multiplication ", py_array*2)
```

```
python array multiplication  [2 4 6]
```

```
import time
start = time.time()
py_list = [i*2 for i in range(1000000)]
print("\n List operation time: ", time.time() - start)

start = time.time()
np_array = np.arange(1000000) * 2
print("\n Numpy operation time: ", time.time() - start)
```

```
 List operation time:  0.06424307823181152

 Numpy operation time:  0.007038593292236328
```

creating array from scratches

```
zeros = np.zeros((3, 4))
print("zeros array: \n", zeros)

ones = np.ones((2, 3))
print("one array: \n", ones)

full = np.full((2, 2), 7)
print("full array: \n", full)

random = np.random.random((2, 3))
print("random array: \n", random)

sequence = np.arange(0, 11, 2)
print("sequnce array: \n", sequence)
```

```
sequence = np.arange(0, 12, 2)
print("sequnce array: \n", sequence)

sequence = np.arange(0, 13, 2)
print("sequnce array: \n", sequence)
```

```
zeros array:
 [[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
one array:
 [[1. 1. 1.]
 [1. 1. 1.]]
full array:
 [[7 7]
 [7 7]]
random array:
 [[0.91637907 0.51155569 0.78545783]
 [0.91104146 0.42468941 0.52789441]]
sequnce array:
 [ 0  2  4  6  8 10]
sequnce array:
 [ 0  2  4  6  8 10]
sequnce array:
 [ 0  2  4  6  8 10 12]
```

vector, matrix and tensor

```
vector = np.array([1, 2, 3])
print("Vector: ", vector)

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print("Matrix: ", matrix)

tensor = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])
print("Tensor: ", tensor)
```

```
Vector:  [1 2 3]
Matrix:  [[1 2 3]
 [4 5 6]]
Tensor:  [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Array properties

```
arr = np.array([[1,2,3],
                [4,5,6],
                [7,8,9],
                [10,11,13]])

print("shape: ", arr.shape)
print("size: ", arr.size)
print("dtype: ", arr.dtype)
print("dimension: ", arr.ndim)
```

```
shape:  (4, 3)
size:  12
dtype:  int64
dimension:  2
```

Array Reshaping

```
arr = np.arange(12)
print("Original array ", arr)

reshaped = arr.reshape((3, 4))
print("\n Reshaped array ", reshaped)

flattened = reshaped.flatten()
print("\n Flattened array ", flattened)
```

```
# ravel (returns view, instead of copy)
raveled = reshaped.ravel()
print("\n raveled array ", raveled)

# Transpose
transpose = reshaped.T
print("\n Transposed array ", transpose)
```

```
Original array  [ 0  1  2  3  4  5  6  7  8  9 10 11]

 Reshaped array  [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

 Flattened array  [ 0  1  2  3  4  5  6  7  8  9 10 11]

 raveled array  [ 0  1  2  3  4  5  6  7  8  9 10 11]

 Transposed array  [[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]
```

PHASE 2:

Numpy Array operations

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Basic Slicing", arr[2:7])
print("With Step", arr[1:8:2])
print("Negative indexing", arr[-3])
```

```
Basic Slicing [3 4 5 6 7]
With Step [2 4 6 8]
Negative indexing 8
```

```
##now work on the 2D array
```

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("Specific element", arr_2d[1, 2])
print("Entire row: ", arr_2d[1]) #row find
print("Entire row: ", arr_2d[:, 1]) #columnn find
```

```
Specific element 6
Entire row:  [4 5 6]
Entire row:  [2 5 8]
```

Sorting

```
unsorted = np.array([3, 1, 4, 1, 5, 9, 2, 6]) # method 1
print("Sorted Array", np.sort(unsorted))

arr_2d_unsorted = np.array([[3, 1], [1, 2], [2, 3]]) #method 2
print("Sorted 2D array by column", np.sort(arr_2d_unsorted, axis=0))
```

```
Sorted Array [1 1 2 3 4 5 6 9]
Sorted 2D array by column [[1 1]
 [2 2]
 [3 3]]
```

Filter

```
numbers = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
even_number = numbers[numbers % 2 == 0]
print("Even numbers", even_number)
```

```
Even numbers [ 2  4  6  8 10]
```

Filter with Mask

```
mask = numbers > 5
print("Numbers greater than 5 ", numbers[mask])
```

```
Numbers greater than 5  [ 6  7  8  9 10]
```

Fancy indexing vs np.where()

```
indices = [0, 2, 4]
print(numbers[indices])

where_result = np.where(numbers > 5)
print(where_result)
print("NP where", numbers[where_result])
```

```
[1 3 5]
(array([5, 6, 7, 8, 9]),)
NP where [ 6  7  8  9 10]
```

```
condition_array = np.array([True, False, True, False, True, False, True, False, True, False])
print(numbers[condition_array])
```

```
[1 3 5 7 9]
```

```
condition_array = np.where(numbers > 5, "true",  "false")
print(condition_array)
```

```
['false' 'false' 'false' 'false' 'false' 'true' 'true' 'true' 'true'
 'true']
```

Adding and removing data

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

combined = np.concatenate((arr1, arr2))
print(combined)
```

```
[1 2 3 4 5 6]
```

array compatibilty

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6, 7])
c = np.array([7, 8, 9])

print("Compatibility shapes", a.shape == b.shape)
```

```
Compatibility shapes False
```

```
original = np.array([[1, 2], [3, 4]])
new_row = np.array([[5, 6]])

with_new_row = np.vstack((original, new_row))
print( "original array: ", original)
print("new array after adding the row: ", with_new_row)

new_col = np.array([[7], [8]])
with_new_col = np.hstack((original, new_col))
print("new array With new column", with_new_col)
```

```
original array:  [[1 2]
 [3 4]]
new array after adding the row:  [[1 2]
 [3 4]
 [5 6]]
new array With new column [[1 2 7]
 [3 4 8]]
```

```
arr = np.array([1, 2, 3, 4, 5])
deleted = np.delete(arr, 2)
```

```
print("Array after deletion: ", deleted)
```

```
Array after deletion:  [1 2 4 5]
```

PHASE 3

==========

NUMPY FOR COMPUTER VISION - ADVANCED BASICS

==========

```
import matplotlib.pyplot as plt
```

1. CREATE A GRAYSCALE IMAGE MATRIX

```
# Here each number represents a pixel brightness
# 0 = black
# 255 = white

image = np.array([
    [50, 80, 120, 200],
    [30, 90, 150, 220],
    [10, 60, 170, 255],
    [0, 40, 130, 240]
])

print("===== ORIGINAL IMAGE MATRIX =====")
print(image)
```

```
===== ORIGINAL IMAGE MATRIX =====
[[ 50  80 120 200]
 [ 30  90 150 220]
 [ 10  60 170 255]
 [  0  40 130 240]]
```

```
Start coding or generate with AI.
```

2. IMAGE SHAPE

```
# shape tells:
# rows = height
# columns = width

print("\n===== IMAGE SHAPE =====")
print(image.shape)
```

```
===== IMAGE SHAPE =====
(4, 4)
```

3. AVERAGE BRIGHTNESS

```
# mean gives average brightness of image

print("\n===== AVERAGE BRIGHTNESS =====")
print(np.mean(image))
```

```
===== AVERAGE BRIGHTNESS =====
115.3125
```

4. MAXIMUM PIXEL VALUE

```
# brightest pixel

print("\n===== BRIGHTEST PIXEL =====")
print(np.max(image))
```

```
===== BRIGHTEST PIXEL =====
255
```

```
Start coding or generate with AI.
```

5. MINIMUM PIXEL VALUE

```
# darkest pixel

print("\n===== DARKEST PIXEL =====")
print(np.min(image))
```

```
===== DARKEST PIXEL =====
0
```

7. IMAGE NORMALIZATION

```
# converting values between 0 and 1
# used heavily in AI and CNNs

normalized_image = image / 255

print("\n===== NORMALIZED IMAGE =====")
print(normalized_image)
```

```
===== NORMALIZED IMAGE =====
[[0.19607843 0.31372549 0.47058824 0.78431373]
 [0.11764706 0.35294118 0.58823529 0.8627451 ]
 [0.03921569 0.23529412 0.66666667 1.        ]
 [0.         0.15686275 0.50980392 0.94117647]]
```

8. IMAGE SLICING (CROPPING)

```
# selecting middle area

cropped = image[1:3, 1:3]

print("\n===== CROPPED IMAGE =====")
print(cropped)
```

```
===== CROPPED IMAGE =====
[[ 90 150]
 [ 60 170]]
```

9. THRESHOLDING

```
# separating the dark and white pixel clearly.
# Your Code
# threshold_image[threshold_image < 100] = 0

# Meaning:

# every pixel less than 100
# becomes black (0)

# So actually:

   # dark areas become MORE black
   # not removed
   # What This Is Doing?

# This is called:

# Thresholding

# We separate:

# dark region
# bright region

threshold_image = image.copy()

threshold_image[threshold_image < 100] = 0
```

```
print("\n===== THRESHOLD IMAGE =====")
print(threshold_image)
```

```
===== THRESHOLD IMAGE =====
[[  0   0 120 200]
 [  0   0 150 220]
 [  0   0 170 255]
 [  0   0 130 240]]
```

10. BOOLEAN MASKING

```
# find bright pixels

bright_pixels = image > 180

print("\n===== BRIGHT PIXEL MASK =====")
print(bright_pixels)
```

```
===== BRIGHT PIXEL MASK =====
[[False False False  True]
 [False False False  True]
 [False False False  True]
 [False False False  True]]
```

11. IMAGE BRIGHTNESS INCREASE

```
# increasing brightness

bright_image = image + 50

# prevent overflow above 255

bright_image = np.clip(bright_image, 0, 255)

print("\n===== BRIGHTNESS INCREASED =====")
print(bright_image)
```

```
===== BRIGHTNESS INCREASED =====
[[100 130 170 250]
 [ 80 140 200 255]
 [ 60 110 220 255]
 [ 50  90 180 255]]
```

12. EDGE DETECTION KERNEL

```
Start coding or generate with AI.
```

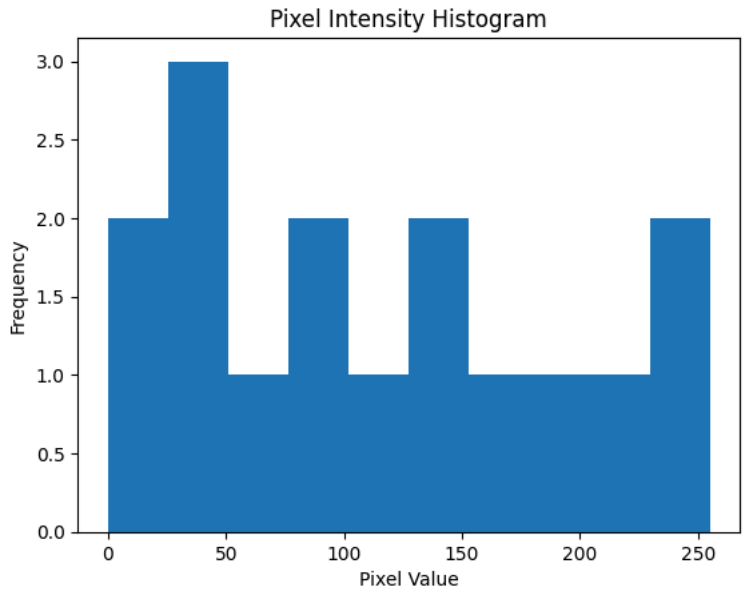
```
# important concept in CV
# this is the Custom Edge Kernel detection methods  This is also called:
# Laplacian-style Edge Kernel.

kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
])

print("\n===== Laplacian EDGE DETECTION KERNEL =====")
print(kernel)
```

```
===== Laplacian EDGE DETECTION KERNEL =====
[[-1 -1 -1]
 [-1  8 -1]
 [-1 -1 -1]]
```

13. HISTOGRAM OF IMAGE

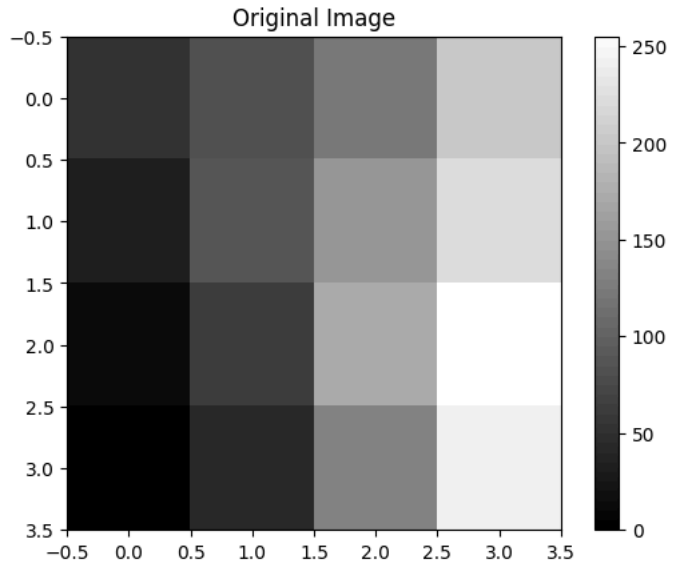
```
# histogram shows brightness distribution

plt.hist(image.flatten())

plt.title("Pixel Intensity Histogram")
```

```
plt.xlabel("Pixel Value")

plt.ylabel("Frequency")

plt.show()
```



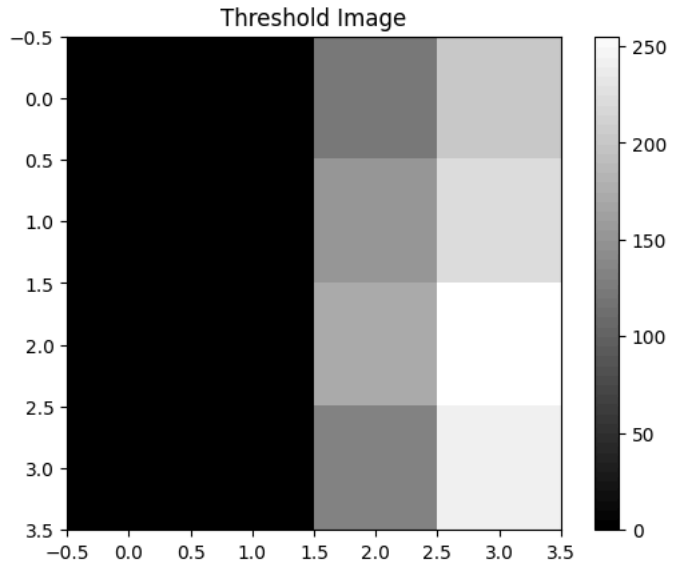
14. DISPLAY IMAGE

```
plt.imshow(image, cmap='gray')

plt.title("Original Image")

plt.colorbar()

plt.show()
```



15. DISPLAY THRESHOLD IMAGE

```
plt.imshow(threshold_image, cmap='gray')

plt.title("Threshold Image")

plt.colorbar()

plt.show()
```



PHASE 4

Some more advanced

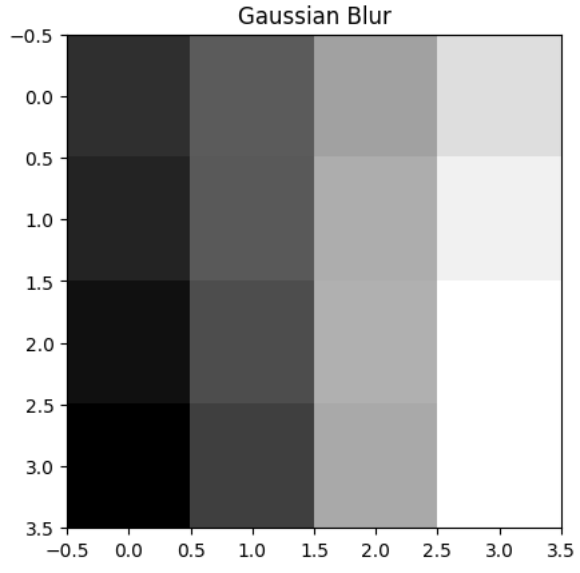
1. GAUSSIAN BLUR (VERY IMPORTANT)

Used for:

noise removal, smoothing image, preprocessing before edge detection

```
from scipy.ndimage import gaussian_filter

blurred = gaussian_filter(image, sigma=1)

print("\n===== GAUSSIAN BLUR =====")
print(blurred)

plt.imshow(blurred, cmap='gray')
plt.title("Gaussian Blur")
plt.show()
```

```
===== GAUSSIAN BLUR =====
[[ 58  88 136 177]
 [ 50  87 144 190]
 [ 37  79 146 200]
 [ 26  69 141 200]]
```

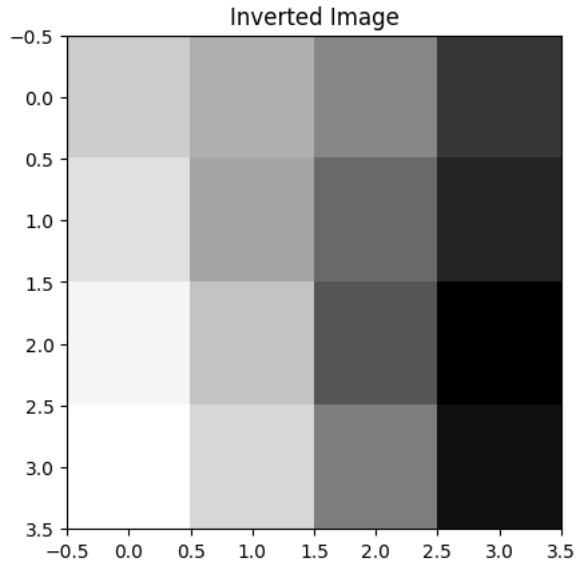


2. IMAGE INVERSION

Frequently asked basic operation.

```
inverted = 255 - image

print("\n===== INVERTED IMAGE =====")
print(inverted)

plt.imshow(inverted, cmap='gray')
plt.title("Inverted Image")
plt.show()
```

```
===== INVERTED IMAGE =====
[[205 175 135  55]
 [225 165 105  35]
 [245 195  85   0]
 [255 215 125  15]]
```



3. IMAGE RESHAPING

VERY IMPORTANT for AI/CNN.

```
reshaped = image.reshape(2, 8)

print("\n===== RESHAPED IMAGE =====")
print(reshaped)
```

```
===== RESHAPED IMAGE =====
[[ 50  80 120 200  30  90 150 220]
 [ 10  60 170 255   0  40 130 240]]
```

4. TRANSPOSE IMAGE

Used in:

matrix operations, image orientation

```
transpose = image.T

print("\n===== TRANSPOSE IMAGE =====")
print(transpose)
```

```
===== TRANSPOSE IMAGE =====
[[ 50  30  10   0]
 [ 80  90  60  40]
 [120 150 170 130]
 [200 220 255 240]]
```

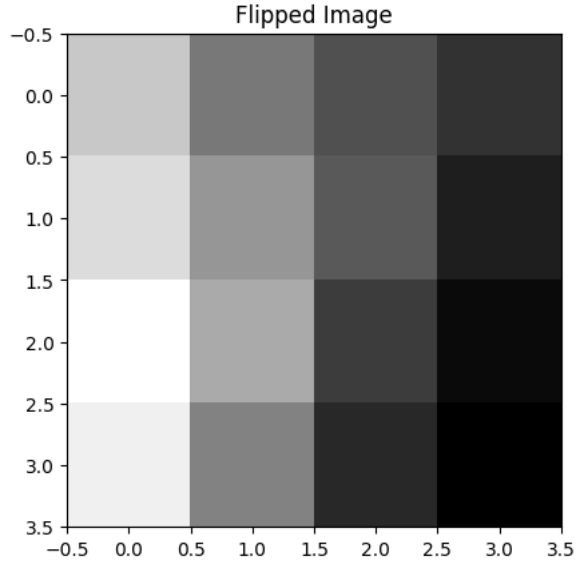
5. FLIPPING IMAGE

VERY COMMON in augmentation.

```
flipped = np.fliplr(image)

print("\n===== FLIPPED IMAGE =====")
```

```
print(flipped)

plt.imshow(flipped, cmap='gray')
plt.title("Flipped Image")
plt.show()
```

```
===== FLIPPED IMAGE =====
[[200 120  80  50]
 [220 150  90  30]
 [255 170  60  10]
 [240 130  40   0]]
```

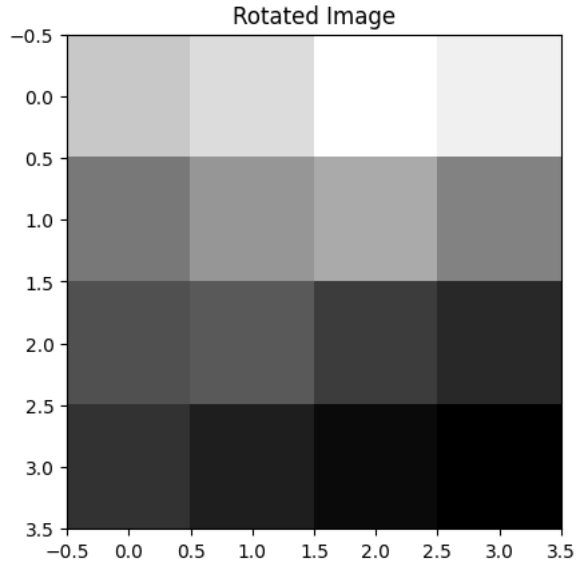


6. ROTATE IMAGE

VERY IMPORTANT.

```
rotated = np.rot90(image)

print("\n===== ROTATED IMAGE =====")
print(rotated)

plt.imshow(rotated, cmap='gray')
plt.title("Rotated Image")
plt.show()
```

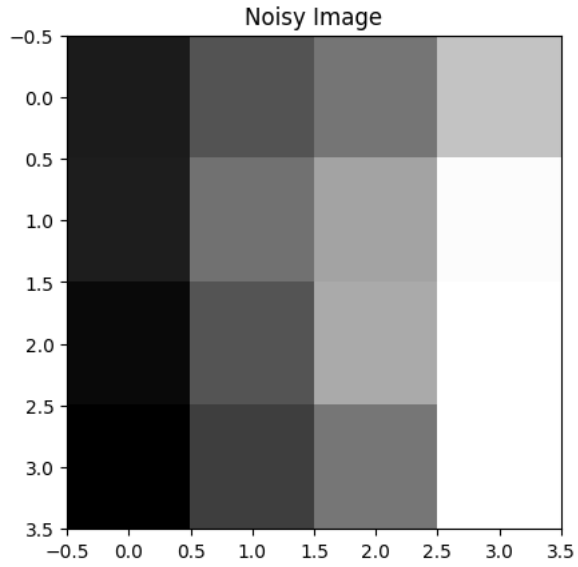
```
===== ROTATED IMAGE =====
[[200 220 255 240]
 [120 150 170 130]
 [ 80  90  60  40]
 [ 50  30  10   0]]
```



7. RANDOM NOISE ADDITION

VERY IMPORTANT in CV research.

```
noise = np.random.randint(0, 50, image.shape)

noisy_image = image + noise

noisy_image = np.clip(noisy_image, 0, 255)

plt.imshow(noisy_image, cmap='gray')
plt.title("Noisy Image")
plt.show()
```



8. HISTOGRAM EQUALIZATION

VERY IMPORTANT concept.

Improves contrast.

```
normalized = (image - np.min(image)) / (np.max(image) - np.min(image))

print("\n===== HISTOGRAM NORMALIZATION =====")
print(normalized)
```

```
===== HISTOGRAM NORMALIZATION =====
[[0.19607843 0.31372549 0.47058824 0.78431373]
 [0.11764706 0.35294118 0.58823529 0.8627451 ]
 [0.03921569 0.23529412 0.66666667 1.        ]
 [0.         0.15686275 0.50980392 0.94117647]]
```

Double-click (or enter) to edit

9. SOBEL EDGE DETECTION

MOST IMPORTANT EDGE METHOD.

```
sobel_x = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
])

print("\n===== SOBEL X KERNEL =====")
print(sobel_x)
```

```
===== SOBEL X KERNEL =====
[[-1  0  1]
 [-2  0  2]
 [-1  0  1]]
```

10. LAPLACIAN KERNEL

```
laplacian = np.array([
    [0, -1, 0],
    [-1, 4, -1],
    [0, -1, 0]
])

print("\n===== LAPLACIAN KERNEL =====")
print(laplacian)
```

```
===== LAPLACIAN KERNEL =====
[[ 0 -1  0]
 [-1  4 -1]
 [ 0 -1  0]]
```

11. RANDOM IMAGE GENERATION

Very common NumPy practice.

```
random_image = np.random.randint(0, 256, (5,5))

print("\n===== RANDOM IMAGE =====")
print(random_image)
```

```
===== RANDOM IMAGE =====
[[ 66 143  89 197 192]
 [ 82 146 133  97 122]
 [254 186  89  72   5]
 [ 64  71 191  92  49]
 [229 192 228 225  65]]
```

12. PIXEL ACCESSING

MOST IMPORTANT beginner concept.

```
print("\n===== PIXEL VALUE =====")
print(image[2,1])
```

```
===== PIXEL VALUE =====
60
```

13. CHANNEL UNDERSTANDING

VERY IMPORTANT for RGB images.

Output meaning:

Height×Width×RGB

```
rgb_image = np.random.randint(0,255,(4,4,3))

print(rgb_image.shape)
```

```
(4, 4, 3)
```